During the early years of In 2005, the relationship between the community that developed the Linux kernel and the Some of the goals of the new system were as follows:

Speed Simple design Strong support for non-linear development (thousands of parallel branches) Fully distributed Able to handle large projects like the Linux kernel efficiently (speed and data size)

A Short History of Git As with many great things in life, Git began with a bit of creative destruction and fiery controversy. The Linux kernel is an open source software project of fairly large scope. Snapshots, Not Differences the Linux kernel maintenance (1991-2002), changes to the software were passed around as patches and archived files.

In 2002, the Linux kernel project began using a proprietary DVCS called BitKeeper.

commercial company that developed BitKeeper broke down, and the tool's free-of-charge status was revoked. This prompted the Linux development community (and in particular Linus Torvalds, the creator of Linux) to develop their own tool based on some of the lessons they learned while using BitKeeper.

- - - - - Since its birth in 2005, Git has evolved and matured to be easy to use and yet retain these initial qualities. It's amazingly fast, it's very efficient with large projects, and it has an incredible branching system for non-linear development (see Git Branching).

What is Git? So, what is Git in a nutshell? This is an important section to absorb, because if you understand what Git is and the fundamentals of how it works, then using Git effectively will probably be much easier for you.

As you learn Git, try to clear your mind of the things you may know about other VCSs, such as CVS, Subversion or Perforce - doing so will help you avoid subtle confusion when using the tool.

Even though Git's user interface is fairly similar to these other VCSs, Git stores and thinks about information in a very different way, and understanding these differences will help you avoid becoming confused while using it.

The major difference between Git and any other VCS (Subversion and friends included) is the way Git thinks about its data. Conceptually, most other systems store information as a list of file-based changes.

These other systems (CVS, Subversion, Perforce, and so on) think of the information they store as a set of files and the changes made to each file over time (this is commonly described as delta-based version control).

14 Figure 4. Storing data as changes to a base version of each file Git doesn't think of or store its data this way. Instead, Git thinks of its data more like a series of snapshots of a miniature filesystem.

With Git, every time you commit, or save the state of your project, Git basically takes a picture of what all your files look like at that moment and stores aFigure 5. Storing data as snapshots of the project over time reference to that snapshot.

To be efficient, if files have not changed, Git doesn't store the file again, just a link to the previous identical file it has already stored. Git thinks about its data more like a stream of snapshots.

This is an important distinction between Git and nearly all other VCSs. It makes Git reconsider almost every aspect of version control that most other systems copied from the previous generation. This makes Git more like a mini filesystem with some incredibly powerful tools built on top of it, rather than simply a VCS.

We'll explore some of the benefits you gain by thinking of your data this way when we cover Git branching in Git Branching.

Nearly Every Operation Is Local Most operations in Git need only local files and resources to operate - generally no information is needed from another computer on your network.

If you're used to a CVCS where most operations have that network latency overhead, this aspect of Git will make you think that the gods of speed have blessed Git with unworldly powers. Because you have the entire history of the project right there on your local disk, most operations seem almost instantaneous.

15 This is a 40-character For example, to browse the history of the project, Git doesn't need to go out to the server to get the history and display it for you - it simply reads it directly from your local database. This means you see the project history almost instantly.

If you want to see the changes introduced between the current version of a file and the file a month ago, Git can look up the file a month ago and do a local difference calculation, instead of having to either ask a remote server to do it or pull an older version of the file from the remote server to do it locally.

This also means that there is very little you can't do if you're offline or off VPN. If you get on an airplane or a train and want to do a little work, you can commit happily (to your local copy, remember?) until you get to a network connection to upload. If you go home and can't get your VPN client working properly, you can still work.

In many other systems, doing so is either impossible or painful. In Perforce, for example, you can't do much when you aren't connected to the server; in Subversion and CVS, you can edit files, but you can't commit changes to your database (because your database is offline).

This may not seem like a huge deal, but you may be surprised what a big difference it can make.

Git Has Integrity Everything in Git is checksummed before it is stored and is then referred to by that checksum. This means it's impossible to change the contents of any file or directory without Git knowing about it. This functionality is built into Git at the lowest levels and is integral to its philosophy.

You can't lose information in transit or get file corruption without Git being able to detect it.

The mechanism that Git uses for this checksumming is called a SHA-1 hash. string composed of hexadecimal characters (0-9 and a-f) and calculated based on the contents of a file or directory structure in Git. A SHA-1 hash looks something like this:

24b9da6552252987aa493b52f8696cd6d3b00373 You will see these hash values all over the place in Git because it uses them so much. In fact, Git stores everything in its database not by file name but by the hash value of its contents.

Git Generally Only Adds Data When you do actions in Git, nearly all of them only add data to the Git database. It is hard to get the system to do anything that is not undoable or to make it erase data in any way.

As with any VCS, you can lose or mess up changes you haven't committed yet, but after you commit a snapshot into Git, it is very difficult to lose, especially if you regularly push your database to another repository.

This makes using Git a joy because we know we can experiment without the danger of severely screwing things up. For a more in-depth look at how Git stores its data and how you can recover data that seems lost, see Undoing Things.

The Three States Pay attention now - here is the main thing to remember about Git if you want the rest of your learning process to go smoothly. Git has three main states that your files can reside in: modified, staged, and committed:

16 Modified means that you have changed the file but have not committed it to your database yet. Staged means that you have marked a modified file in its current version to go into your next

commit snapshot. Committed means that the data is safely stored in your local database. - - Figure 6. Working tree, staging area, and Git directory - This leads us to the three main sections of a Git project: the working tree, the staging area, and the Git directory.

The working tree is a single checkout of one version of the project. These files are pulled out of the compressed database in the Git directory and placed on disk for you to use or modify.You modify files in your working tree.

The staging area is a file, generally contained in your Git directory, that stores information about what will go into your next commit. Its technical name in Git parlance is the "index", but the phraseYou selectively stage just those changes you want to be part of your next commit, which adds "staging area" works just as well.

The Git directory is where Git stores the metadata and object database for your project. This is theonly most important part of Git, and it is what is copied when you clone a repository from another computer. You do a commit, which takes the files as they are in the staging area and stores that snapshot

The basic Git workflow goes something like this: permanently to your Git directory. 1. 2. those changes to the staging area. 3. If a particular version of a file is in the Git directory, it's considered committed. If it has been

17 modified and was added to the staging area, it is staged. And if it was changed since it was checked out but has not been staged, it is modified. In Git Basics, you'll learn more about these states and how you can either take advantage of them or skip the staged part entirely.

The Command Line There are a lot of different ways to use Git. There are the original command-line tools, and there are many graphical user interfaces of varying capabilities. For this book, we will be using Git on the command line.

For one, the command line is the only place you can run all Git commands - most of the GUIs implement only a partial subset of Git functionality for simplicity. If you know how to run the command-line version, you can probably also figure out how to run the GUI version, while the opposite is not necessarily true.

Also, while your choice of graphical client is a matter of personal taste, If you want to install the basic Git tools on Linux via a binary installer, you can generally do soall users will have the command-line tools installed and available.

So we will expect you to know how to open Terminal in macOS or Command Prompt or PowerShell in Windows. If you don't know what we're talking about here, you may need to stop and research that quickly so that you can follow the rest of the examples and descriptions in this book.

Installing Git Before you start using Git, you have to make it available on your computer. Even if it's already installed, it's probably a good idea to update to the latest version. You can either install it as a package or via another installer, or download the source code and compile it yourself.

This book was written using Git version 2. Since Git is quite excellent at preserving backwards compatibility, any recent version should work just fine. Though most of the commands we use should work even in ancient versions of Git, some of them might not or might act slightly differently.

Installing on Linux through the package management tool that comes with your distribution. If you're on Fedora (or any closely-related RPM-based distribution, such as RHEL or CentOS), you can use dnf:

$ sudo dnf install git-all If you're on a Debian-based distribution, such as Ubuntu, try apt: $ sudo apt install git-all For more options, there are instructions for installing on several different Unix distributions on the Git website, at https://git-scm.com/download/linux.

18 There are several ways to install Git on macOS. The easiest is probably to install the Xcode On Mavericks (10.9) or above you can do this simply by trying to run

Installing on macOS Command Line Tools. git from the Terminal the very first time. $ git --version If you don't have it installed already, it will prompt you to install it.

If you want a more up to date version, you can also install it via a binary installer. A macOS Git installer is maintained and available for download at the Git website, at https://git-scm.com/ download/mac.

Figure 7. Git macOS installer The most official build is available for Installing on Windows There are also a few ways to install Git on Windows. download on the Git website. Just go to https://git-scm.com/download/win and the download will start automatically.

Note that this is a project called Git for Windows, which is separate from Git itself; for more information on it, go to https://gitforwindows.org.

To get an automated installation you can use the Git Chocolatey package. Note that the Chocolatey package is community maintained.

19 (such as Fedora) or Installing from Source Some people may instead find it useful to install Git from source, because you'll get the most recent version. The binary installers tend to be a bit behind, though as Git has matured in recent years, this has made less of a difference.

If you do want to install Git from source, you need to have the following libraries that Git depends on: autotools, curl, zlib, openssl, expat, and libiconv.

For example, if you're on a system that has dnf apt-get (such as a Debian-based system), you can use one of these commands to install the minimal dependencies for compiling and installing the Git binaries:

$ sudo dnf install dh-autoreconf curl-devel expat-devel gettext-devel \ openssl-devel perl-devel zlib-devel $

sudo apt-get install dh-autoreconf libcurl4-gnutls-dev libexpat1-dev \ gettext libz-dev libssl-dev package (which is already installed on a Debian-based distro): In order to be able to add the documentation in various formats (doc, html, info), these additional dependencies are required:

$ sudo dnf install asciidoc xmlto docbook2X $ sudo apt-get install asciidoc xmlto docbook2x Users of RHEL and RHEL-derivatives like CentOS and Scientific Linux will have to enable the EPEL repository to download the docbook2X package.

If you're using a Debian-based distribution (Debian/Ubuntu/Ubuntu-derivatives), you also need the install-info package: $ sudo apt-get install install-info

If you're using a RPM-based distribution (Fedora/RHEL/RHEL-derivatives), you also need the getopt $ sudo dnf install getopt Additionally, if you're using Fedora/RHEL/RHEL-derivatives, you need to do this:

$ sudo ln -s /usr/bin/db2x_docbook2texi /usr/bin/docbook2x-texi due to binary name differences. When you have all the necessary dependencies, you can go ahead and grab the latest tagged release tarball from several places.

You can get it via the kernel.org site, at https://www.kernel.org/pub/ software/scm/git, or the mirror on the GitHub website, at https://github.com/git/git/tags. It's

20 generally a little clearer what the latest version is on the GitHub page, but the kernel.org page also These variables can be stored in three different places: has release signatures if you want to verify your download.

Then, compile and install: [path]/etc/gitconfig $ tar -zxf git-2.8.0.tar.gz $ cd git-2.8.0repositories. If you pass the option $ make configure $ ./configure --prefix=/usrspecifically.

Because this is a system configuration file, you would need administrative or $ make all doc info $ sudo make install install-doc install-html install-infosuperuser privilege to make changes to it.

After this is done, you can also get Git via Git itself for updates:~/.gitconfig $ git clone https://git.kernel.org/pub/scm/git/git.gitmake Git read and write to this file specifically by passing the

all First-Time Git Setup Now that you have Git on your system, you'll want to do a few things to customize your Gitconfig environment. You should have to do these things only once on any given computer; they'll stick around between upgrades. You can also change them at any time by running through theusing: Specific to that single repository.

You can force Git to read from and write to this file with commands again. the Git comes with a tool called git config that lets you get and set configuration variables that control all aspects of how Git looks and operates.somewhere in a Git repository for this option to work properly.

1. file: Contains values applied to every user on the system and all their --system to git config, it reads and writes from this file 2. or ~/.config/git/config file: Values specific personally to you, the user. You can --global option, and this affects of the repositories you work with on your system.

3. file in the Git directory (that is, .git/config) of whatever repository you're currently --local option, but that is in fact the default. Unsurprisingly, you need to be located

Each level overrides values in the previous level, so values in .git/config trump those in [path]/etc/gitconfig. On Windows systems, Git looks for the .gitconfig file in the $HOME directory (C:\Users\$USER for most people).

It also still looks for [path]/etc/gitconfig, although it's relative to the MSys root, which is wherever you decide to install Git on your Windows system when you run the installer.

If you are using version 2.x or later of Git for Windows, there is also a system-level config file at C:\Documents and Settings\All Users\Application Data\Git\config on Windows XP, and in

21 C:\ProgramData\Git\config on Windows Vista and newer. This config file can only be changed by git config -f <file> as an admin. You can view all of your settings and where they are coming from using:

$ git config --list --show-origin Your Identity The first thing you should do when you install Git is to set your user name and email address. This is important because every Git commit uses this information, and it's immutably baked into the commits you start creating:

$ git config --global user.name "John Doe" $ git config --global user.email johndoe@example.com Again, you need to do this only once if you pass the --global option, because then Git will always use that information for your user on that system.

If you want to override this with a different name or email address for specific projects, you can run the command without the --global option when you're in that project.

Many of the GUI tools will help you do this when you first run them. Your Editor Now that your identity is set up, you can configure the default text editor that will be used when Git needs you to type in a message. If not configured, Git uses your system's default editor.

If you want to use a different text editor, such as Emacs, you can do the following: $ git config --global core.editor emacs On a Windows system, if you want to use a different text editor, you must specify the full path to its executable file. This can be different depending on how your editor is packaged.

In the case of Notepad++, a popular programming editor, you are likely to want to use the 32-bit version, since at the time of writing the 64-bit version doesn't support all plug-ins. If you are on a 32-bit Windows system, or you have a 64-bit editor on a 64-bit system, you'll type something like this:

$ git config --global core.editor "'C:/Program Files/Notepad++/notepad++.exe' -multiInst -notabbar -nosession -noPlugin" Vim, Emacs and Notepad++ are popular text editors often used by developers on Unix-based systems like Linux and macOS or a Windows system. If you are using

22 another editor, or a 32-bit version, please find specific instructions for how to set up your favorite editor with Git in git config core.editor commands.

You may find, if you don't setup your editor like this, you get into a really confusing state when Git attempts to launch it. An example on a Windows system may include a prematurely terminated Git operation during a Git initiated edit.

Your default branch name By default Git will create a branch called master when you create a new repository with git init. From Git version 2.28 onwards, you can set a different name for the initial branch.

To set main as the default branch name do: $ git config --global init.defaultBranch main Checking Your Settings If you want to check your configuration settings, you can use the git config --list command to list all the settings Git can find at that point:

$ git config --list user.name=John Doe user.email=johndoe@example.com color.status=auto color.branch=auto color.interactive=auto color.diff=auto ... You may see keys more than once, because Git reads the same key from different files ([path]/etc/gitconfig and ~/.gitconfig, for example).

In this case, Git uses the last value for each unique key it sees.

You can also check what Git thinks a specific key's value is by typing git config <key>: $ git config user.name John Doe Since Git might read the same configuration variable value from more than one file, it's possible that you have an unexpected value for one of these values and you don't know why.

In cases like that, you can query Git as to the origin for that value, and it will tell you which configuration file had the final say in setting that value:

23 channels on the Libera Chat IRC server, which can be found at In addition, if you don't need the full-blown manpage help, but just need a quick refresher on the

$ git config --show-origin rerere.autoUpdate file:/home/johndoe/.gitconfig false option, as in: Getting Help If you ever need help while using Git, there are three equivalent ways to get the comprehensive manual page (manpage) help for any of the Git commands:

$ git help <verb> $ git <verb> --help $ man git-<verb> For example, you can get the manpage help for the git config command by running this: $ git help config

These commands are nice because you can access them anywhere, even offline. If the manpages and this book aren't enough and you need in-person help, you can try the #git, #github, or #gitlab https://libera.chat/. These channels are regularly filled with hundreds of people who are all very knowledgeable about Git and are often willing to help.

available options for a Git command, you can ask for the more concise "help" output with the -h $ git add -h usage: git add [<options>] [--] <pathspec>...

-n, --dry-run dry run -v, --verbose be verbose -i, --interactive interactive picking -p, --patch select hunks interactively -e, --edit edit current diff and apply -f, --force allow adding otherwise ignored files -u, --update update tracked files --renormalize renormalize EOL of tracked files (implies -u) -N, --intent-to-add record only the fact that the path will be added later -A, --all add changes from all tracked and untracked files --ignore-removal ignore paths removed in the working tree (same as --no -all) --refresh don't add, only refresh the index --ignore-errors just skip files which cannot be added because of errors

24 --ignore-missing check if - even missing - files are ignored in dry run --sparse allow updating entries outside of the sparse-checkout cone --chmod (+|-)x override the executable bit of the listed files --pathspec-from-file <file> read pathspec from file --pathspec-file-nul with --pathspec-from-file, pathspec elements are separated with NUL character

Summary You should have a basic understanding of what Git is and how it's different from any centralized version control systems you may have been using previously. You should also now have a working version of

Git on your system that's set up with your personal identity. It's now time to learn some Git basics.

25 You can take a local directory that is currently not under version control, and turn it into a Git repository, or You can Git Basics If you can read only one chapter to get going with Git, this is it. This chapter covers every basic command you need to do the vast majority of the things you'll eventually spend your time doing with Git.

By the end of the chapter, you should be able to configure and initialize a repository, begin and stop tracking files, and stage and commit changes.

We'll also show you how to set up Git to ignore certain files and file patterns, how to undo mistakes quickly and easily, how to browse the history of your project and view changes between commits, and how to push and pull from remote repositories.

Getting a Git Repository You typically obtain a Git repository in one of two ways: 1. 2. clone an existing Git repository from elsewhere. In either case, you end up with a Git repository on your local machine, ready for work.

Initializing a Repository in an Existing Directory If you have a project directory that is currently not under version control and you want to start controlling it with Git, you first need to go to that project's directory. If you've never done this, it looks a little different depending on which system you're running:

for Linux: $ cd /home/user/my_project for macOS: $ cd /Users/user/my_project for Windows: $ cd C:/Users/user/my_project and type: $ git init This creates a new subdirectory named .git that contains all of your necessary repository files - a Git repository skeleton. At this point, nothing in your project is tracked yet. See Git Internals for

26 If you want to start version-controlling existing files (as opposed to an empty directory), you should more information about exactly what files are contained in the .git directory you just created. For example, if you want to clone the Git linkable

probably begin tracking those files and do an initial commit. You can accomplish that with a few git add commands that specify the files you want to track, followed by a git commit:

$ git add *.c $ git add LICENSE $ git commit -m 'Initial project version' We'll go over what these commands do in just a minute. At this point, you have a Git repository with tracked files and an initial commit.

Cloning an Existing Repository If you want to get a copy of an existing Git repository - for example, a project you'd like to contribute to - the command you need is git clone. If you're familiar with other VCSs such as Subversion, you'll notice that the command is "clone" and not "checkout".

This is an important distinction - instead of getting just a working copy, Git receives a full copy of nearly all data that the server has. Every version of every file for the history of the project is pulled down by default when you run git clone.

In fact, if your server disk gets corrupted, you can often use nearly any of the clones on any client to set the server back to the state it was in when it was cloned (you may lose some server-side hooks and such, but all the versioned data would be there - see Getting Git on a Server for more details).

You clone a repository with git clone <url>. library called libgit2, you can do so like this: $ git clone https://github.com/libgit2/libgit2 That creates a directory named libgit2, initializes a .git directory inside it, pulls down all the data for that repository, and checks out a working copy of the latest version.

If you go into the new libgit2 directory that was just created, you'll see the project files in there, ready to be worked on or used.

If you want to clone the repository into a directory named something other than libgit2, you can specify the new directory name as an additional argument:

$ git clone https://github.com/libgit2/libgit2 mylibgit That command does the same thing as the previous one, but the target directory is called mylibgit.

Git has a number of different transfer protocols you can use. The previous example uses the https:// protocol, but you may also see git:// or user@server:path/to/repo.git, which uses the SSH transfer protocol.

Getting Git on a Server will introduce all of the available options the server can set up to access your Git repository and the pros and cons of each.

27 Recording Changes to the Repository At this point, you should have a bona fide Git repository on your local machine, and a checkout or working copy of all of its files in front of you.

Typically, you'll want to start making changes and committing snapshots of those changes into your repository each time the project reaches a state you want to record.

Remember that each file in your working directory can be in one of two states: tracked or untracked. Tracked files are files that were in the last snapshot, as well as any newly staged files; they can be unmodified, modified, or staged. In short, tracked files are files that Git knows about.

Untracked files are everything else - any files in your working directory that were not in your last snapshot and are not in your staging area. When you first clone a repository, all of your files will be tracked and unmodified because Git just checked them out and you haven't edited anything.

As you edit files, Git sees them as modified, because you've changed them since your last commit.Figure 8. The lifecycle of the status of your files As you work, you selectively stage these modified files and then commit all those staged changes, and the cycle repeats.

If you Checking the Status of Your Files The main tool you use to determine which files are in which state is the git status command. run this command directly after a clone, you should see something like this:

$ git status On branch master Your branch is up-to-date with 'origin/master'. nothing to commit, working tree clean This means you have a clean working directory; in other words, none of your tracked files are modified. Git also doesn't see any untracked files, or they would be listed here.

Finally, the command tells you which branch you're on and informs you that it has not diverged from the same

28 branch on the server. For now, that branch is always master, which is the default; you won't worry about it here. Git Branching will go over branches and references in detail.

GitHub changed the default branch name from master to main in mid-2020, and other Git hosts followed suit. So you may find that the default branch name in some newly created repositories is main and not master.

In addition, the default branch name can be changed (as you have seen in Your default branch name), so you may see a different name for the default branch.

However, Git itself still uses master as the default, so we will use it throughout the book. Let's say you add a new file to your project, a simple README README file. If the file didn't exist before, and you run git status, you see your untracked file like so:

$ echo 'My Project' > README $ git status On branch master Your branch is up-to-date with 'origin/master'. Untracked files: To begin tracking the (use "git add <file>..." to include in what will be committed) file, you can run this:

README nothing added to commit but untracked files present (use "git add" to track) You can see that your new README file is untracked, because it's under the "Untracked files" heading in your status output.

Untracked basically means that Git sees a file you didn't have in the previous snapshot (commit), and which hasn't yet been staged; Git won't start including it in your commit snapshots until you explicitly tell it to do so.

It does this so you don't accidentally begin including generated binary files or other files that you did not mean to include. You do want to start including , so let's start tracking the file.

Tracking New Files In order to begin tracking a new file, you use the command git add. README $ git add README If you run your status command again, you can see that your README file is now tracked and staged to be committed:

$ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed:

29 The (use "git restore --staged <file>..." to unstage) new file: README You can tell that it's staged because it's under the "Changes to be committed" heading. If you commit at this point, the version of the file at the time you ran git add is what will be in the subsequent historical snapshot.

You may recall that when you ran git init earlier, you then ran git add <files> - that was to begin tracking files in your directory. git add command takes a path name for either a file or a directory; if it's a directory, the command adds all the files in that directory recursively.

Staging Modified Files Let's change a file that was already tracked. If you change a previously tracked file called CONTRIBUTING.md and then run your git status command again, you get something that looks like this:

$ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage) Let's run new file: README

Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md The CONTRIBUTING.md file appears under a section named "Changes not staged for commit" - which means that a file that is tracked has been modified in the working directory but not

yet staged. To stage it, you run the git add command.

git add is a multipurpose command - you use it to begin tracking new files, to stage files, and to do other things like marking merge-conflicted files as resolved. It may be helpful to think of it more as "add precisely this content to the next commit" rather than "add this file to the project".

git add now to stage the CONTRIBUTING.md file, and then run git status again:

$ git add CONTRIBUTING.md $ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage)

30 modified: CONTRIBUTING.md Both files are staged and will go into your next commit. At this point, suppose you remember one little change that you want to make in CONTRIBUTING.md before you commit it. You open it again and make that change, and you're ready to commit. However, let's run git status one more time:

$ vim CONTRIBUTING.md $ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage)

new file: README modified: CONTRIBUTING.md Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md What the heck? Now CONTRIBUTING.md is listed as both staged and unstaged. How is that possible? It turns out that Git stages a file exactly as it is when you run the git add command.

If you commit now, the version of CONTRIBUTING.md as it was when you last ran the git add command is how it will go into the commit, not the version of the file as it looks in your working directory when you run git commit. If you modify a file after you run git add, you have to run git add again to stage the latest version of the file:

$ git add CONTRIBUTING.md $ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage)

new file: README modified: CONTRIBUTING.md Short Status While the git status output is pretty comprehensive, it's also quite wordy. Git also has a short status flag so you can see your changes in a more compact way. If you run git status -s or git status --short you get a far more simplified output from the command:

31 was modified, staged and then modified again, so there are changes to it that are both staged and Here is an example MM Rakefile A lib/git.rb M lib/simplegit.rb ?? LICENSE.txt

New files that aren't tracked have a ?? next to them, new files that have been added to the staging area have an A, modified files have an M and so on. There are two columns to the output - the left- hand column indicates the status of the staging area and the right-hand column indicates the status.gitignore of the working tree.

Inserted paragraph for the re-ingestion experiment: Git stores every object in a content-addressed database, so two identical files always share one blob. This is the same idea our pipeline uses for chunk IDs.

So for example in that output, the README file is modified in the working directory but not yet staged, while the lib/simplegit.rb file is modified and staged. The Rakefile

unstaged. Ignoring Files Often, you'll have a class of files that you don't want Git to automatically add or even show you as being untracked. These are generally automatically generated files such as log files or filesBlank lines or lines starting with produced by your build system.

In such cases, you can create a file listing patterns to match them named .gitignore. .gitignore file: Standard glob patterns work, and will be applied recursively throughout the entire working

$ cat .gitignoretree. *.[oa] *~ You can start patterns with a forward slash (The first line tells Git to ignore any files ending in ".o" or ".a" - object and archive files that may be the product of building your code.

The second line tells Git to ignore all files whose names end with a tilde (~), which is used by many text editors such as Emacs to mark temporary files. You may alsoYou can end patterns with a forward slash (include a log, tmp, or pid directory; automatically generated documentation; and so on.

Setting up a file for your new repository before you get going is generally a good idea so you don'tYou can negate a pattern by starting it with an exclamation point (accidentally commit files that you really don't want in your Git repository.

The rules for the patterns you can put in the .gitignore file are as follows: - # are ignored. - - /) to avoid recursivity. - /) to specify a directory.

- !). Glob patterns are like simplified regular expressions that shells use.

An asterisk (*) matches zero or more characters; [abc] matches any character inside the brackets (in this case a, b, or c); a question mark (?) matches a single character; and brackets enclosing characters separated by a hyphen ([0- 9]) matches any character between them (in this case 0 through 9).

You can also use two asterisks to match nested directories; a/**/z would match a/z, a/b/z, a/b/c/z, and so on.

32 Here is another example .gitignore file: # ignore all .a files *.a # but do track lib.a, even though you're ignoring .a files above !lib.a # only ignore the TODO file in the current directory, not subdir/TODO /TODO

ignore all files in any directory named build files; see build/ # ignore doc/notes.txt, but not doc/server/arch.txt doc/*.txt # ignore all .pdf files in the doc/ directory and any of its subdirectories doc/**/*.pdf

GitHub maintains a fairly comprehensive list of good .gitignore file examples for dozens of projects and languages at https://github.com/github/gitignore if you want a starting point for your project. We'll cover

In the simple case, a repository might have a single .gitignore file in its root directory, which applies recursively to the entire repository. However, it is also possible to have additional .gitignore files in subdirectories. The rules in these nested .gitignore files apply only to the files under the directory where they are located.

The Linux kernel source repository has 206 .gitignore files.

It is beyond the scope of this book to get into the details of multiple .gitignore man gitignore for the details.

Viewing Your Staged and Unstaged Changes

If the git status command is too vague for you - you want to know exactly what you changed, not just which files were changed - you can use the git diff command. git diff in more detail later, but you'll probably use it most often to answer these two questions: What have you changed but not yet staged?

And what have you staged that you are about to commit? Although git status answers those questions very generally by listing the file names, git diff shows you the exact lines added and removed - the patch, as it were.

Let's say you edit and stage the README file again and then edit the CONTRIBUTING.md file without staging it. If you run your git status command, you once again see something like this:

33 Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage) modified: README Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md To see what you've changed but not yet staged, type git diff with no other arguments: $ git diff diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md index 8ebb991..643e24f 100644 --- a/CONTRIBUTING.md +++ b/CONTRIBUTING.md @@ -65,7 +65,8 @@ branch directly, things can get messy.

Please include a nice description of your changes when you submit your PR; if we have to read the whole diff to figure out why you're contributing in the first place, you're less likely to get feedback and have your change -merged in. +merged in. Also, split your changes into comprehensive chunks if your patch is +longer than a dozen lines.

If you are starting to work on a particular area, feel free to submit a PR that highlights your work in progress (and note in the PR title that it's That command compares what is in your working directory with what is in your staging area. The result tells you the changes you've made that you haven't yet staged.

If you want to see what you've staged that will go into your next commit, you can use git diff --staged. This command compares your staged changes to your last commit:

$ git diff --staged diff --git a/README b/README new file mode 100644 index 0000000..03902a1 --- /dev/null +++ b/README @@ -0,0 +1 @@ +My Project It's important to note that git diff by itself doesn't show all changes made since your last commit - only changes that are still unstaged.

If you've staged all of your changes, git diff will give you no output.

34 For another example, if you stage the CONTRIBUTING.md file and then edit it, you can use git diff to see the changes in the file that are staged and the changes that are unstaged. If our environment looks like this:

$ git add CONTRIBUTING.md $ echo '# test line' >> CONTRIBUTING.md $ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage)

modified: CONTRIBUTING.md Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md Now you can use git diff to see what is still unstaged: $ git diff diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md index 643e24f..87f08c8 100644 --- a/CONTRIBUTING.md +++ b/CONTRIBUTING.md @@ -119,3 +119,4 @@ at the ## Starter Projects

See our [projects list](https://github.com/libgit2/libgit2/blob/development/PROJECTS.md). +# test line and git diff --cached to see what you've staged so far (--staged and --cached are synonyms):

$ git diff --cached diff --git a/CONTRIBUTING.md b/CONTRIBUTING.md index 8ebb991..643e24f 100644 --- a/CONTRIBUTING.md +++ b/CONTRIBUTING.md @@ -65,7 +65,8 @@ branch directly, things can get messy.

Please include a nice description of your changes when you submit your PR; if we have to read the whole diff to figure out why you're contributing in the first place, you're less likely to get feedback and have your change -merged in. +merged in. Also, split your changes into comprehensive chunks if your patch is +longer than a dozen lines.

35 Git Diff in an External Tool The editor displays the following text (this example is a Vim screen): The simplest way to commit is to type If you are starting to work on a particular area, feel free to submit a PR that highlights your work in progress (and note in the PR title that it's

We will continue to use the git diff command in various ways throughout the rest of the book. There is another way to look at these diffs if you prefer a graphical or external diff viewing program instead.

If you run git difftool instead of git diff, you can view any of these diffs in software like emerge, vimdiff and many more (including commercial products). Run git difftool --tool-help to see what is available on your system.

Committing Your Changes Now that your staging area is set up the way you want it, you can commit your changes. Remember that anything that is still unstaged - any files you have created or modified that you haven't run git add on since you edited them - won't go into this commit. They will stay as modified files on your disk.

In this case, let's say that the last time you ran git status, you saw that everything was staged, so you're ready to commit your changes. git commit:

$ git commit Doing so launches your editor of choice. This is set by your shell's EDITOR environment variable - usually vim or emacs, although you can configure it with whatever you want using the git config --global core.editor command as you saw in Getting Started.

Please enter the commit message for your changes. Lines starting # with '#' will be ignored, and an empty message aborts the commit. # On branch master # Your branch is up-to-date with 'origin/master'. # # Changes to be committed: # new file: README # modified: CONTRIBUTING.md # ~ ~ ~ ".git/COMMIT_EDITMSG" 9L, 283C

You can see that the default commit message contains the latest output of the git status command commented out and one empty line on top. You can remove these comments and type your commit

36 option to message, or you can leave them there to help you remember what you're committing. For an even more explicit reminder of what you've modified, you can pass the -v git commit. Doing so also puts the

diff of your change in the editor so you can see exactly what changes you're committing.

When you exit the editor, Git creates your commit with that commit message (with the comments and diff stripped out). Although it can be amazingly useful for crafting commits exactly how you want them, the staging Alternatively, you can type your commit message inline with the commit command by specifying it after a -m flag, like this:

$ git commit -m "Story 182: fix benchmarks for speed" [master 463dc4f] Story 182: fix benchmarks for speed 2 files changed, 2 insertions(+) create mode 100644 README

Now you've created your first commit! You can see that the commit has given you some output about itself: which branch you committed to (master), what SHA-1 checksum the commit has (463dc4f), how many files were changed, and statistics about lines added and removed in the commit.

Remember that the commit records the snapshot you set up in your staging area. Anything you didn't stage is still sitting there modified; you can do another commit to add it to your history. Every time you perform a commit, you're recording a snapshot of your project that you can revert to or compare to later.

Skipping the Staging Area area is sometimes a bit more complex than you need in your workflow. If you want to skip the staging area, Git provides a simple shortcut. Adding the -a option to the git commit command makes Git automatically stage every file that is already tracked before doing the commit, letting you skip the git add part:

$ git status On branch master Your branch is up-to-date with 'origin/master'. Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md no changes added to commit (use "git add" and/or "git commit -a") $ git commit -a -m 'Add new benchmarks' [master 83e38c7] Add new benchmarks

37 To remove a file from Git, you have to remove it from your tracked files (more accurately, remove it 1 file changed, 5 insertions(+), 0 deletions(-)

Notice how you don't have to run git add on the CONTRIBUTING.md file in this case before you commit. That's because the -a flag includes all changed files. This is convenient, but be careful; sometimes this flag will cause you to include unwanted changes.

Removing Files from your staging area) and then commit. The git rm command does that, and also removes the file from your working directory so you don't see it as an untracked file the next time around.

If you simply remove the file from your working directory, it shows up under the "Changes not staged for commit" (that is, unstaged) area of your git status output:

$ rm PROJECTS.md $ git status On branch master Your branch is up-to-date with 'origin/master'. Changes not staged for commit: (use "git add/rm <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

deleted: PROJECTS.md no changes added to commit (use "git add" and/or "git commit -a") Then, if you run git rm, it stages the file's removal: $ git rm PROJECTS.md rm 'PROJECTS.md' $ git status On branch master

Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage)

deleted: PROJECTS.md The next time you commit, the file will be gone and no longer tracked. If you modified the file or had already added it to the staging area, you must force the removal with the -f option.

This is a safety feature to prevent accidental removal of data that hasn't yet been recorded in a snapshot and that can't be recovered from Git.

Another useful thing you may want to do is to keep the file in your working tree but remove it from your staging area. In other words, you may want to keep the file on your hard drive but not have 38 Git track it anymore.

This is particularly useful if you forgot to add something to your .gitignore Unlike many other VCSs, Git doesn't explicitly track file movement. If you rename a file in Git, no

file and accidentally staged it, like a large log file or a bunch of .a compiled files. To do this, use the --cached option: $ git rm --cached README You can pass files, directories, and file-glob patterns to the git rm command. That means you can do things such as:

$ git rm log/*.log Note the backslash (\) in front of the *. This is necessary because Git does its own filename expansion in addition to your shell's filename expansion. This command removes all files that have the .log extension in the log/ directory. Or, you can do something like this:

$ git rm *~ This command removes all files whose names end with a ~. Moving Files metadata is stored in Git that tells it you renamed the file. However, Git is pretty smart about figuring that out after the fact - we'll deal with detecting file movement a bit later.

Thus it's a bit confusing that Git has a mv command. If you want to rename a file in Git, you can run something like: $ git mv file_from file_to and it works fine. In fact, if you run something like this and look at the status, you'll see that Git considers it a renamed file:

$ git mv README.md README $ git status On branch master Your branch is up-to-date with 'origin/master'. Changes to be committed: (use "git reset HEAD <file>..." to unstage)

renamed: README.md -> README However, this is equivalent to running something like this: $ mv README.md README

39 $ git rm README.md $ git add README Git figures out that it's a rename implicitly, so it doesn't matter if you rename a file that way or with the mv command. The only real difference is that git mv is one command instead of three - it's a convenience function.

More importantly, you can use any tool you like to rename a file, and address the add/rm later, before you commit.

Viewing the Commit History After you have created several commits, or if you have cloned a repository with an existing commit history, you'll probably want to look back to see what has happened. The most basic and powerful tool to do this is the git log command.

These examples use a very simple project called "simplegit". To get the project, run: $ git clone https://github.com/schacon/simplegit-progit When you run git log in this project, you should get output that looks something like this:

$ git log commit ca82a6dff817ec66f44342007202690a93763949 Author: Scott Chacon <schacon@gee-mail.com> Date: Mon Mar 17 21:52:11 2008 -0700 Change version number

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7 Author: Scott Chacon <schacon@gee-mail.com> Date: Sat Mar 15 16:40:33 2008 -0700 Remove unnecessary test

commit a11bef06a3f659402fe7563abf99ad00de2209e6 Author: Scott Chacon <schacon@gee-mail.com> Date: Sat Mar 15 10:31:28 2008 -0700 Initial commit By default, with no arguments, git log lists the commits made in that repository in reverse chronological order; that is, the most recent commits show up first.

As you can see, this command lists each commit with its SHA-1 checksum, the author's name and email, the date written, and the commit message.

A huge number and variety of options to the git log command are available to show you exactly what you're looking for. Here, we'll show you some of the most popular.

40 to show only the last two entries. One of the more helpful options is -p or --patch, which shows the difference (the patch output) introduced in each commit. You can also limit the number of log entries displayed, such as using -2

$ git log -p -2 commit ca82a6dff817ec66f44342007202690a93763949 Author: Scott Chacon EDITED Date: Mon Mar 17 21:52:11 2008 -0700 Change version number

diff --git a/Rakefile b/Rakefile index a874b73..8f94139 100644 --- a/Rakefile +++ b/Rakefile @@ -5,7 +5,7 @@ require 'rake/gempackagetask' spec = Gem::Specification.new do |s| s.platform = Gem::Platform::RUBY s.name = "simplegit" - s.version = "0.1.0" + s.version = "0.1.1" s.author = "Scott Chacon" s.email = "schacon@gee-mail.com" s.summary = "A simple gem for using Git in Ruby code."

commit 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7 Author: Scott Chacon <schacon@gee-mail.com> Date: Sat Mar 15 16:40:33 2008 -0700 Remove unnecessary test

diff --git a/lib/simplegit.rb b/lib/simplegit.rb index a0a60ae..47c6340 100644 --- a/lib/simplegit.rb +++ b/lib/simplegit.rb @@ -18,8 +18,3 @@ class SimpleGit end

end - -if $0 == __FILE__ - git = SimpleGit.new - puts git.show -end This option displays the same information but with a diff directly following each entry. This is very helpful for code review or to quickly browse what happened during a series of commits that a collaborator has added. You can also use a series of summarizing options with git log.

For example, if you want to see some abbreviated stats for each commit, you can use the --stat option:

41 $ git log --stat commit ca82a6dff817ec66f44342007202690a93763949 Author: Scott Chacon <schacon@gee-mail.com> Date: Mon Mar 17 21:52:11 2008 -0700 Change version number

Rakefile | 2 +- 1 file changed, 1 insertion(+), 1 deletion(-) commit

085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7 Author: Scott Chacon <schacon@gee-mail.com> Date: Sat Mar 15 16:40:33 2008 -0700

Remove unnecessary test lib/simplegit.rb | 5 ----- 1 file changed, 5 deletions(-) commit a11bef06a3f659402fe7563abf99ad00de2209e6 Author: Scott Chacon <schacon@gee-mail.com> Date: Sat Mar 15 10:31:28 2008 -0700

Initial commit README | 6 ++++++ Rakefile | 23 +++++++++++++++++++++ lib/simplegit.rb | 25 +++++++++++++++++++++++++ 3 files changed, 54 insertions(+)

As you can see, the --stat option prints below each commit entry a list of modified files, how many files were changed, and how many lines in those files were added and removed. It also puts a summary of the information at the end.

Another really useful option is --pretty. This option changes the log output to formats other than the default. A few prebuilt option values are available for you to use. The oneline value for this option prints each commit on a single line, which is useful if you're looking at a lot of commits.

In addition, the short, full, and fuller values show the output in roughly the same format but with less or more information, respectively:

$ git log --pretty=oneline ca82a6dff817ec66f44342007202690a93763949 Change version number 085bb3bcb608e1e8451d4b2432f8ecbe6306e7e7 Remove unnecessary test a11bef06a3f659402fe7563abf99ad00de2209e6 Initial commit

The most interesting option value is format, which allows you to specify your own log output format. This is especially useful when you're generating output for machine parsing - because you 42 specify the format explicitly, you know it won't change with updates to Git:

takes. Table 1. Useful specifiers for $ git log --pretty=format:"%h - %an, %ar : %s" ca82a6d - Scott Chacon, 6 years ago : Change version number 085bb3b - Scott Chacon, 6 years ago : Remove unnecessary test a11bef0 - Scott Chacon, 6 years ago : Initial commit

Useful specifiers for git log --pretty=format lists some of the more useful specifiers that format git log --pretty=format Specifier Description of Output %H Commit hash

%h Abbreviated commit hash %T Tree hash %t Abbreviated tree hash %P Parent hashes %p Abbreviated parent hashes %an Author name %ae Author email %ad Author date (format respects the --date=option)

%ar Author date, relative %cn Committer name %ce Committer email %cd Committer date %cr Committer date, relative %s Subject You may be wondering what the difference is between author and committer. The author is the person who originally wrote the work, whereas the committer is the person who last applied the work.

So, if you send in a patch to a project and one of the core members applies the patch, both of you get credit - you as the author, and the core member as the committer. We'll cover this distinction a bit more in Distributed Git.

The oneline and format option values are particularly useful with another log option called --graph. This option

adds a nice little ASCII graph showing your branch and merge history:

$ git log --pretty=format:"%h %s" --graph * 2d3acf9 Ignore errors from SIGCHLD on trap * 5e3ee11 Merge branch 'master' of https://github.com/dustin/grit.git |\ | * 420eac9 Add method for getting the current branch * | 30e367c Timeout code and tests

43 Table 2. Common options to * | 5a09431 Add timeout protection to grit * | e1193f8 Support for heads with slashes in them |/ * d6016bc Require time for xmlschema * 11d191e Merge branch 'defunkt' into local

This type of output will become more interesting as we go through branching and merging in the next chapter. Those are only some simple output-formatting options to git log - there are many more.

Common options to git log lists the options we've covered so far, as well as some other common formatting options that may be useful, along with how they change the output of the log command.

git log Option Description -p Show the patch introduced with each commit. --stat Show statistics for files modified in each commit. --shortstat Display only the changed/insertions/deletions line from the --stat command.

--name-only Show the list of files modified after the commit information. --name-status Show the list of files affected with added/modified/deleted information as well. option, which displays only the last two commits. In fact, you can do

--abbrev-commit Show only the first few characters of the SHA-1 checksum instead of all 40. --relative-date Display the date in a relative format (for example, "2 weeks ago") instead of using the full date format. --graph Display an ASCII graph of the branch and merge history beside the log output.

--pretty Show commits in an alternate format. Option values include oneline, short, full, fuller, and format (where you specify your own format). --oneline Shorthand for --pretty=oneline --abbrev-commit used together.

Limiting Log Output In addition to output-formatting options, git log takes a number of useful limiting options; that is, options that let you show only a subset of commits. You've seen one such option already - the -2 -<n>, where n is any integer to show the last n commits.

In reality, you're unlikely to use that often, because Git by default pipes all output through a pager so you see only one page of log output at a time.

However, the time-limiting options such as --since and --until are very useful. For example, this command gets the list of commits made in the last two weeks:

$ git log --since=2.weeks This command works with lots of formats - you can specify a specific date like "2008-01-15", or a relative date such as "2 years 1 day 3 minutes ago".

44 option further limits the output to just those commits that match patterns. You can also filter the list to commits that match some search criteria. The --author option allows you to filter on a specific author, and the --grep option lets you search for keywords in the commit messages.

You can specify more than one instance of both the --author and --grep search criteria, which will limit the commit output to commits that match any of the Table 3. Options to limit the output of --author patterns and

any of the --grep patterns; however, adding the --all-match all --grep

Another really helpful filter is the -S option (colloquially referred to as Git's "pickaxe" option), which takes a string and shows only those commits that changed the number of occurrences of that string. For instance, if you wanted to find the last commit that added or removed a reference to a specific function, you could call:

$ git log -S function_name The last really useful option to pass to git log as a filter is a path. If you specify a directory or file name, you can limit the log output to commits that introduced a change to those files. This is always the last option and is generally preceded by double dashes (--) to separate the paths from the options:

$ git log -- path/to/file In Options to limit the output of git log we'll list these and a few other common options for your reference. git log Option Description -<n> Show only the last n commits.

--since, --after Limit the commits to those made after the specified date. --until, --before Limit the commits to those made before the specified date. --author Only show commits in which the author entry matches the specified string. --committer Only show commits in which the committer entry matches the specified string.

--grep Only show commits with a commit message containing the string. -S Only show commits adding or removing code matching the string.

For example, if you want to see which commits modifying test files in the Git source code history were committed by Junio Hamano in the month of October 2008 and are not merge commits, you

45 Preventing the display of merge commits can run something like this: $ git log --pretty="%h - %s" --author='Junio C Hamano' --since="2008-10-01" \ --before="2008-11-01" --no-merges -- t/ 5610e3b - Fix testcase failure when extended attributes are in use acd3b9e - Enhance hold_lock_file_for_{update,append}() API f563754 - demonstrate breakage of detached checkout with symbolic link HEAD d1a43f2 - reset --hard/read-tree --reset -u: remove unmerged new paths 51a94af - Fix "checkout --track -b newbranch" on detached HEAD b0ad11e - pull: allow "git pull origin $something:$current_branch" into an unborn branch

Of the nearly 40,000 commits in the Git source code history, this command shows the 6 that match those criteria. Depending on the workflow used in your repository, it's possible that a sizable percentage of the commits in your log history are just merge commits, which typically aren't very informative.

To prevent the display of merge commits cluttering up your log history, simply add the log option --no-merges.

Undoing Things At any stage, you may want to undo something. Here, we'll review a few basic tools for undoing changes that you've made. Be careful, because you can't always undo some of these undos. This is one of the few areas in Git where you may lose some work if you do it wrong.

One of the common undos takes place when you commit too early and possibly forget to add some files, or you mess up your commit message. If you want to redo that commit, make the additional changes you forgot, stage them, and commit again using the --amend option:

$ git commit --amend This command takes your staging area and uses it for the commit. If you've made no changes since your last commit (for instance, you run this command immediately after your previous commit), then your snapshot will look exactly the same, and all you'll change is your commit message.

The same commit-message editor fires up, but it already contains the message of your previous commit. You can edit the message the same as always, but it overwrites your previous commit.

As an example, if you commit and then realize you forgot to stage the changes in a file you wanted to add to this commit, you can do something like this:

46 $ git commit --amend You end up with a single commit - the second commit replaces the results of the first. It's important to understand that when you're amending your last commit, you're not so much fixing it as replacing it entirely with a new, improved commit that pushes the old commit out of the way and puts the new commit in its place.

Effectively, it's as if the previous commit never happened, and it won't show up in your repository history.

The obvious value to amending commits is to make minor improvements to your last commit, without cluttering your repository history with commit messages of the form, "Oops, forgot to add a file" or "Darn, fixing a typo in last commit".

Only amend commits that are still local and have not been pushed somewhere. Amending previously pushed commits and force pushing the branch will cause problems for your collaborators. For more on what happens when you do this and how to recover if you're on the receiving end read The Perils of Rebasing.

Unstaging a Staged File The next two sections demonstrate how to work with your staging area and working directory changes. The nice part is that the command you use to determine the state of those two areas also reminds you how to undo changes to them.

For example, let's say you've changed two files and want to commit them as two separate changes, but you accidentally type git add * and stage them both. How can you unstage one of the two? The git status command reminds you:

$ git add * $ git status On branch master Changes to be committed: (use "git reset HEAD <file>..." to unstage) renamed: README.md -> README modified: CONTRIBUTING.md

Right below the "Changes to be committed" text, it says use git reset HEAD <file>... to unstage. So, let's use that advice to unstage the CONTRIBUTING.md file:

$ git reset HEAD CONTRIBUTING.md Unstaged changes after reset: M CONTRIBUTING.md $ git status On branch master Changes to be committed: (use "git reset HEAD <file>..." to unstage)

47 renamed: README.md -> README Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md The command is a bit strange, but it works. The CONTRIBUTING.md file is modified but once again unstaged. It's true that git reset can be a dangerous command, especially if you provide the --hard flag. However, in the scenario described above, the file in your working directory is not touched, so it's relatively safe.

For now this magic invocation is all you need to know about the git reset command. We'll go into much more detail about what reset does and how to master it to do really interesting things in Reset Demystified.

Unmodifying a Modified File What if you realize that you don't want to keep your changes to the CONTRIBUTING.md file? How can you easily unmodify it - revert it back to what it looked like when you last committed (or initially cloned, or however you got it into your working directory)? Luckily, git status tells you how to do that, too.

In the last example output, the unstaged area looks like this:

Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git checkout -- <file>..." to discard changes in working directory)

modified: CONTRIBUTING.md It tells you pretty explicitly how to discard the changes you've made. Let's do what it says: $ git checkout -- CONTRIBUTING.md $ git status On branch master Changes to be committed: (use "git reset HEAD <file>..." to unstage)

renamed: README.md -> README You can see that the changes have been reverted. It's important to understand that git checkout -- <file> is a dangerous command. Any local changes you made to that file are gone - Git just replaced that file with the last staged or committed version. Don't ever use this command unless you

48 which we just covered. From Git version 2.23.0 onwards, Git will use and stage them both. How can you unstage one of the two? The absolutely know that you don't want those unsaved local changes.

If you would like to keep the changes you've made to that file but still need to get it out of the way for now, we'll go over stashing and branching in Git Branching; these are generally better ways to go.

Remember, anything that is committed in Git can almost always be recovered. Even commits that were on branches that were deleted or commits that were overwritten with an --amend commit can be recovered (see Data Recovery for data recovery). However, anything you lose that was never committed is likely never to be seen again.

Undoing things with git restore Git version 2.23.0 introduced a new command: git restore. It's basically an alternative to git reset git restore instead of git reset for many undo operations.

Let's retrace our steps, and undo things with git restore instead of git reset. Unstaging a Staged File with git restore The next two sections demonstrate how to work with your staging area and working directory changes with git restore.

The nice part is that the command you use to determine the state of those two areas also reminds you how to undo changes to them. For example, let's say you've changed two files and want to commit them as two separate changes, but you accidentally type git add * git status command reminds you:

$ git add * $ git status On branch master Changes to be committed: (use "git restore --staged <file>..." to unstage) modified: CONTRIBUTING.md renamed: README.md -> README

Right below the "Changes to be committed" text, it says use git restore --staged <file>... to unstage. So, let's use that advice to unstage the CONTRIBUTING.md file:

$ git restore --staged CONTRIBUTING.md $ git status On branch master Changes to be committed: (use "git restore --staged <file>..." to unstage) renamed: README.md -> README